

Attack of Killer Microseconds

The author argues that the existing systems optimize latency for millisecond and nanosecond timescale events but lack support for μs events that are becoming important in datacenters. Processors use prefetching, out of order execution and branch prediction for low level interactions with nanosecond scale devices. Operating systems use software based techniques like context switching for millisecond scale events. OS context switching during long waits (several milliseconds) for disk reads often outweigh context switching overhead of a few μs . These synchronous models for interacting with nanosecond and millisecond-scale devices are easier than the alternative of asynchronous models.

The author discusses the emergence of low latency IO devices that operate in the μs scale. For instance, in datacenter networking, the time taken by a packet to traverse the length of the datacenter is approximately 1 μs . Likewise the latency of the emerging NVMe storage devices is also in the same ballpark. In addition to that, there is a growing need to exploit lower latency storage and communication due to the demise of Dennard scaling and Moore's law. The existing hardware and software techniques to deal with nanosecond and millisecond latencies does not scale well for μs latencies.

The author discusses how a 2 μs operation can easily be stretched to over a hundred μs due to RDMA costs, dispatch overheads from network thread to a different processor's operation head, relying on interrupts rather than polling and use of TCP/IP stack instead of RPC stack. There are many other latencies that stack up, for example, queuing overheads, and several hundred milliseconds of overhead to wake a processor core in sleep state. Similar observations have been made for NVMe storage, which suffers from up to a 5x overhead over the base (few μs) due to kernel interrupt handling and other delays. The author suggests that the hardware and software stacks must be rethought for these μs scale latencies.

μs scale events also negatively effect the processor efficiency. Simple queueing theory suggests that as service time increases, the throughput decreases. In the μs regime, service time is mostly composed of overheads, negatively impacting the throughput. The paper suggests that a 16 μs overhead can decrease the processor efficiency by 40%. These overheads include the serialization and deserialization of data, memory allocation and deallocation, network stack costs, compression and encryption. This adds to the killer μs challenge. Many of these overheads cannot be offloaded to a separate core or accelerator as IO operations tend to be closely coupled with the main work on the processor.

The author suggests some solutions to this problem like designing μs aware software and new hardware mechanisms to extend the use of synchronous blocking mechanisms and thread level parallelism to the μs range.

I found the paper to be quite intriguing as one would think that a design that deals with nanoseconds should not have a problem dealing with order of μs , however, the situation is quite the contrary. The efficiency impact on both network and processors is an interesting insight. It may be interesting to explore the impact of using higher link speeds that are now commercially available on the transmission time and its impact on the network and processors.

Cloud Programming Simplified: Berkeley View on Serverless Computing

The paper is an follow up of sorts to the Berkeley View on Cloud Computing paper that we discussed earlier. The paper explores the pros and cons of serverless computing which has gained popularity over the last few years. The author references the six advantages of cloud computing listed in the paper and builds a case for serverless computing. The paper argues that EC2 initially gained popularity because it allowed users to recreate the same computing environment as their local computer, however, having to manage the VMs can be cumbersome for many users. Due to these needs, AWS introduced Lambda, similar to cloud functions on GCP, where the user writes the code and leaves the rest to the cloud provider. The author defines serverless computing as a combination of functions as a service and backend as a service.

The author compares serverful to writing assembly code while serverless is like writing Python code, and makes three critical distinctions between the two. Firstly, in serverless computing, computation and storage are decoupled and scale independently. Storage is provided by a separate cloud service while computation is stateless. Secondly, in serverless computing, the user does not have to manage the resources. The user just has to upload the code. Thirdly, in serverless, the user pays in proportion to resources used rather than resources allocated. Serverless is an abstraction between the base cloud platform and applications. The paper also compares serverless computing to its predecessors like the app engine, and to services like kubernetes which simplifies the management of serverful computing. Compared to its predecessors, serverless computing offers better autoscaling, strong isolation, platform flexibility and service ecosystem support. On the other hand, the major difference between kubernetes and serverless computing is the pricing model.

Serverless computing is attractive for cloud providers, customers and researchers alike. For cloud providers, it allows allocation of unused or old resources while offering increased productivity and cost saving for the customer. It also allows for architectural innovation. For instance if ARM or RISC-V offer better cost performance than x86, the switch is easier with serverless. For researchers, it opens new avenues of research. The paper also lists several limitations of serverless computing. In the five use-cases tested by the authors, serverless was more considerably more expensive in three of them albeit only being significantly slower in one. Apart from that there are also other challenges with serverless computing, for instance lack of adequate storage for fine grained operations, lack of fine grained coordination, poor performance for standard communication patterns and initialization overhead.

Having worked with AWS extensively myself, I believe the paper is on point when it comes to the advantages and disadvantages of cloud computing. The author has covered all of the objections I had in my mind. For instance, the author talks about high latency with databases and object stores like S3, but also covered the workarounds like redis and the cost issue associated with them. Nevertheless, the importance of serverless computing in today's landscape cannot be denied as companies today run products primarily on serverless computing.